

SIMATS ENGINEERING
Department of Computer Science & Engineering
ASSESSMENT TOOL 3- CLASS TEST

Course Code:	CSA1205	Course Title:	Computer Architecture for Multicore Systems
CO Assessed:	C05	Assessment:	ASSESSMENT TOOL3- CLASS TEST
Weightage:	30%	Total Marks:	25
C05 Statement:	<i>Construct I/O communication mechanisms by comparing memory-mapped and I/O-mapped techniques, interrupt handling (vectored, hardware, software), DMA controllers, and advanced bus interfaces including PCIe, USB, NVMe, and Thunderbolt.</i>		
Student Name:	SHAIK HAZIPEERA	Reg.No.:	192572056
Department:	CSE(AI)	Date:	

ANNEXURE A
CLASS TEST

1. Design a heterogeneous I/O bus architecture comparing USB, PCIe, and Thunderbolt, and construct a scalable communication model for multiple peripherals.
2. Construct an interrupt handling framework for a multi-core processor comparing vectored interrupts and shared interrupt lines, and analyze latency impact.
3. Design a high-speed NVMe storage subsystem using PCIe lanes, DMA transfers, and memory-mapped registers, and justify performance improvements.
4. Construct an I/O scheduling mechanism comparing cycle stealing DMA and burst mode DMA for continuous multimedia streaming.
5. Design a multi-level I/O communication architecture combining memory-mapped I/O, interrupt handling, and advanced bus interfaces, and evaluate throughput.

Code of Conduct:

I SHAIK HAZIPEERA (192572056) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

S. Hazipeera

Signature of the student:

MARKS ALLOCATION

	Conceptual Understanding (30%)	Accuracy of Answers (30%)	Problem Solving & Application Skills (20%)	Presentation & Organization (10%)	Completion & Time Management (10%)
Q1					
Q2					
Q3					
Q4					
Q5					

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Conceptual Understanding	30%	Demonstrates excellent understanding of all concepts with accurate explanations and applications	Shows good understanding of concepts with minor errors	Shows basic understanding but with some misconceptions	Shows limited understanding with major conceptual errors
Accuracy of Answers	30%	Provides completely correct answers with proper steps, logic, and final solutions	Provides mostly correct answers with few minor mistakes	Provides partially correct answers with missing steps/errors	Provides incorrect or incomplete answers with minimal accuracy
Problem Solving & Application Skills	20%	Applies appropriate methods/techniques effectively and solves complex problems independently	Applies suitable methods with minor guidance/errors	Attempts problem solving but has difficulty applying concepts correctly	Unable to apply concepts or select suitable methods
Presentation & Organization	10%	Answers are well structured, neat, and clearly presented with proper notation/format	Answers are organized with minor presentation issues	Answers lack proper organization and clarity in some areas	Poorly organized answers with unclear presentation
Completion & Time Management	10%	Completes all questions within the allotted time with effective planning	Completes most questions within time	Attempts only part of the test due to time management issues	Very few questions attempted or incomplete submission

I/O Bus Architecture & Interfacing — Answers

Computer Organization & Architecture — Long Answer Set (8–10 Marks each)

Q1. Design a heterogeneous I/O bus architecture comparing USB, PCIe, and Thunderbolt, and construct a scalable communication model for multiple peripherals.

Overview

A heterogeneous I/O bus architecture connects peripherals of different speed classes to the CPU through a hierarchy of buses, each chosen for the traffic pattern it serves best.

Feature	USB (3.2/4)	PCIe (Gen4/5)	Thunderbolt (4/5)
Topology	Tiered star (host + hubs)	Point-to-point, switched	Daisy-chain over PCIe+DP tunnel
Max Throughput	~20 Gbps	~64 GB/s (x16 Gen5)	~80 Gbps (120 Gbps asym.)
Latency	Higher (polled/host-driven)	Very low (direct memory access)	Low (tunnels native PCIe)
Typical Use	Keyboards, storage, phones	GPU, NVMe SSD, NIC	External GPU, displays, docks
Protocol Overhead	Moderate	Low	Low–Moderate

Scalable Communication Model

A layered model lets many heterogeneous peripherals share the CPU–memory path without contention:

- Root Complex / Host Controller: single entry point that maps every bus type into a unified physical address space.
- Switch/Hub Layer: PCIe switches and USB hubs fan out to multiple downstream ports, enabling tree-structured scaling.
- Bridge Layer: USB and Thunderbolt controllers act as PCIe endpoints, so all traffic ultimately funnels through PCIe lanes to memory.
- Arbitration: a bus arbiter assigns bandwidth using priority/weighted round-robin so high-speed PCIe devices are not starved by slow USB traffic.
- Address/Interrupt Mapping: each device gets a unique memory-mapped region and MSI-X interrupt vector, allowing thousands of peripherals to be added without bus redesign.

Justification

This model scales because adding a peripheral only requires attaching it to the nearest switch/hub — the root complex and memory bus are untouched. PCIe's point-to-point links avoid the collision domain problem of a shared bus, while USB/Thunderbolt tunneling reuses PCIe's bandwidth guarantees, giving a single coherent, expandable I/O fabric.

[8–10 Marks]

Q2. Construct an interrupt handling framework for a multi-core processor comparing vectored interrupts and shared interrupt lines, and analyze latency impact.

Framework Design

In a multi-core system, an Advanced Programmable Interrupt Controller (APIC/IO-APIC) sits between devices and cores. It receives all device interrupts, decides which core services each one, and delivers it via a local APIC on the target core.

- Device raises IRQ → I/O-APIC/interrupt controller logs the source.
- Controller resolves the interrupt to a vector number and target core (fixed, round-robin, or load-based routing).

- Local APIC on that core signals the CPU, which pushes context and jumps directly to the vector's ISR entry in the Interrupt Vector Table.
- ISR executes, sends End-Of-Interrupt (EOI), and control returns to the interrupted task.

Vectored vs Shared Interrupt Lines

Aspect	Vectored Interrupts	Shared Interrupt Lines
Identification	Device supplies its own vector; direct ISR jump	One line for many devices; handler must poll each
Latency	Low — $O(1)$ dispatch	Higher — $O(n)$ polling overhead
Scalability	Good with MSI/MSI-X (thousands of vectors)	Limited by number of physical IRQ lines
Hardware Cost	Slightly higher (vector table, MSI logic)	Lower — legacy, simple wiring
Core Contention	Can target a specific core directly	May need broadcast + software filtering

Latency Impact Analysis

Vectored interrupts (especially MSI-X) give near-constant latency because the ISR address is known immediately, and per-core vectors avoid inter-core signalling delay. Shared lines add latency proportional to the number of devices sharing the line, since the OS must invoke each device driver's handler in turn until the interrupting device is found. On an n-core system, poor routing (e.g., always interrupting core 0) also creates a hotspot that increases queuing latency; hence modern designs prefer MSI-X with per-core vector tables and affinity-based routing to keep worst-case latency bounded.

[8-10 Marks]

Q3. Design a high-speed NVMe storage subsystem using PCIe lanes, DMA transfers, and memory-mapped registers, and justify performance improvements.

Subsystem Design

- Physical Link: NVMe SSD connects via PCIe (typically x4 lanes, Gen4/5), giving dedicated, full-duplex, point-to-point bandwidth to the root complex.
- Command Interface: memory-mapped controller registers (BAR0) expose Submission Queue (SQ) and Completion Queue (CQ) doorbell registers directly in the CPU's physical address space.
- Queuing Model: NVMe supports up to 64K queues with 64K commands each — the driver writes a command to an SQ in host memory, then "rings the doorbell" via a memory-mapped write.
- Data Movement: the SSD controller uses DMA (bus-mastering) to read/write data directly between host memory and flash, without CPU copying each word.
- Completion: controller writes a completion entry to the CQ and raises an MSI-X interrupt on the requesting core.

Performance Justification

Design Choice	Performance Benefit
PCIe lanes (x4, Gen4/5)	High raw bandwidth (~8 GB/s per x4 Gen4 link), low latency, no shared-bus contention
Memory-mapped registers	CPU issues commands with simple load/store instructions — no slow port I/O
Multiple parallel queues	Removes single-queue bottleneck; each CPU core has its own SQ/CQ pair, enabling true parallel I/O

DMA transfer	Frees CPU from data copying, overlapping computation with I/O transfer
MSI-X per-queue interrupts	Completion signalled directly to the issuing core, cutting interrupt latency

Together these eliminate the three main bottlenecks of legacy SATA/AHCI storage — single command queue, programmed I/O overhead, and shared bus latency — giving NVMe SSDs their order-of-magnitude higher IOPS and throughput.

[8-10 Marks]

Q4. Construct an I/O scheduling mechanism comparing cycle stealing DMA and burst mode DMA for continuous multimedia streaming.

Scheduling Mechanism

For continuous multimedia streaming (audio/video), the DMA controller (DMAC) must move large, steady data streams from a peripheral (capture card, disk) to memory while the CPU keeps rendering/decoding on time. A scheduler decides, cycle by cycle, whether the bus is granted to the CPU or the DMAC.

- Buffered Double-Queue Model: two ping-pong buffers — while one is filled by DMA, the CPU consumes the other, avoiding stalls.
- DMAC requests the bus via HOLD/HLDA (or bus-request/grant) signals; the scheduler decides grant timing based on the chosen mode below.

Cycle Stealing DMA vs Burst Mode DMA

Aspect	Cycle Stealing DMA	Burst Mode DMA
Bus Usage	Takes one bus cycle at a time, alternating with CPU	Takes the bus for the entire block transfer at once
CPU Impact	Minimal — CPU only stalls for single cycles	CPU fully halted until burst completes
Throughput	Lower — overhead of repeated arbitration	Higher — no re-arbitration between words
Best Suited For	Real-time/low-latency tasks needing CPU responsiveness	Large, time-critical block transfers (e.g., video frame)
Streaming Fit	Good for interleaving audio with UI/decoding tasks	Good for bulk frame transfer where jitter-free burst is fine

Recommendation for Multimedia Streaming

A hybrid scheduler works best: use burst mode to move large video frame buffers quickly during vertical-blank windows (when CPU has no display-critical work), and cycle stealing for continuous, low-latency audio samples that must not starve the CPU. This keeps frame throughput high while preserving smooth, glitch-free audio and UI responsiveness.

[8-10 Marks]

Q5. Design a multi-level I/O communication architecture combining memory-mapped I/O, interrupt handling, and advanced bus interfaces, and evaluate throughput.

Architecture (3 Levels)

- Level 1 — Device/Register Level: peripheral control and status registers are mapped into the CPU's physical address space (memory-mapped I/O), so devices are accessed with ordinary load/store instructions.
- Level 2 — Event/Notification Level: instead of the CPU polling registers, devices signal completion or errors via interrupts (vectored/MSI-X), letting the CPU do useful work between events.
- Level 3 — Transport/Bus Level: an advanced bus interface (PCIe hierarchy, with USB/Thunderbolt tunneled through it) physically carries the memory-mapped transactions and interrupt messages between device and CPU/memory, with switches providing scalable fan-out.

How the Levels Combine

A CPU write to a memory-mapped register (Level 1) is transparently converted into a bus transaction (Level 3) and carried to the device; the device's completion or error condition is reported back as an interrupt message (Level 2), itself just a special memory write (MSI) on the same bus. This unifies control, notification, and data transport into one consistent transaction model.

Throughput Evaluation

Layer	Contribution to Throughput
Memory-mapped I/O	Removes special I/O instruction overhead; register access is as fast as a memory access
Interrupts (MSI-X)	Eliminates polling overhead, freeing bus/CPU cycles for actual data transfer
Advanced bus (PCIe switch fabric)	Point-to-point, full-duplex lanes give near-linear throughput scaling as lanes/devices are added
Combined effect	Bottleneck shifts from bus/CPU overhead to raw physical bandwidth, so effective throughput approaches the theoretical link bandwidth

Overall, layering memory-mapped access, interrupt-driven notification, and a switched high-speed bus produces a system where CPU overhead per I/O operation is minimal and aggregate throughput scales with the number of PCIe lanes/devices rather than being capped by a single shared, polled bus — a major improvement over older isolated I/O with polling.

[8-10 Marks]